5. Missionaries and Cannibals Problem

```
from collections import deque

# State representation: (missionaries_left, cannibals_left, boat_position)
# boat_position: 1 for Left bank, 0 for Right bank
START = (3, 3, 1)
GOAL = (0, 0, 0)

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    # Check if cannibals outnumber missionaries on either bank
    if (m > 0 and m < c) or ((3 - m) > 0 and (3 - m) < (3 - c)):
        return False
    return True

def get_successors(state):
    m, c, boat = state
    successors = []
    # Possible moves: (M, C) to put in the boat
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    for dm, dc in moves:
        if boat == 1:  # Moving Left to Right
            next_state = (m - dm, c - dc, 0)
        else:        # Moving Right to Left
            next_state = (m + dm, c + dc, 1)

        if is_valid(next_state[0], next_state[1]):
            successors.append(next_state)
```

```
    return successors

def solve_missionaries_cannibals():
    queue = deque([[START]])
    visited = {START}

    while queue:
        path = queue.popleft()
        current = path[-1]

        if current == GOAL:
            return path

        for next_state in get_successors(current):
            if next_state not in visited:
                visited.add(next_state)
                queue.append(path + [next_state])
    return None

if __name__ == "__main__":
    solution = solve_missionaries_cannibals()
    if solution:
        print("Successfully found a path to the goal state:")
        for step, state in enumerate(solution):
            bank = "Left" if state[2] == 1 else "Right"
            print(f"Step {step}: Left Bank (M:{state[0]}, C:{state[1]}) | Boat is on: {bank}")
    else:
        print("No solution found.")
```

main.py Visualize Run

```python
from collections import deque
START = (3, 3, 1)
GOAL = (0, 0, 0)

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
    if (m > 0 and m < c) or ((3 - m) > 0 and (3 - m) < (3 - c)):
        return False
    return True

def get_successors(state):
    m, c, boat = state
    successors = []
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    for dm, dc in moves:
        if boat == 1:
```

Output

```
Successfully found a path to the goal state:
Step 0: Left Bank (M:3, C:3) | Boat is on: Left
Step 1: Left Bank (M:3, C:1) | Boat is on: Right
Step 2: Left Bank (M:3, C:2) | Boat is on: Left
Step 3: Left Bank (M:3, C:0) | Boat is on: Right
Step 4: Left Bank (M:3, C:1) | Boat is on: Left
Step 5: Left Bank (M:1, C:1) | Boat is on: Right
Step 6: Left Bank (M:2, C:2) | Boat is on: Left
Step 7: Left Bank (M:0, C:2) | Boat is on: Right
Step 8: Left Bank (M:0, C:3) | Boat is on: Left
Step 9: Left Bank (M:0, C:1) | Boat is on: Right
Step 10: Left Bank (M:1, C:1) | Boat is on: Left
Step 11: Left Bank (M:0, C:0) | Boat is on: Right
```